Reprodução e Avaliação do GraphIDS: Detecção Auto-Supervisionada de Intrusão em Rede com Grafos

Rafael Mendes Bezerra Xavier, Danilo Regino Barrote, Bernardo Siqueira Batista e Felipe Maran Alves
Centro de Informática, Universidade Federal de Pernambuco, Recife, Brasil
CIN0114 – Técnicas de Ataque e Detecção de Intrusão, 2026.2 – Equipe 10

***Resumo*—NIDS supervisionados dependem de tráfego rotulado e falham diante de ataques novos; métodos auto-supervisionados com redes neurais de grafos, como o Anomal-E, dispensam rótulos, mas separam o aprendizado de representações da detecção. O GraphIDS (Guerra *et al.*, NeurIPS 2025) une as duas etapas: um E-GraphSAGE gera embeddings de fluxos NetFlow com contexto topológico e um autoencoder Transformer com máscara de atenção os reconstrói, treinado só com tráfego benigno; o erro de reconstrução é o score de anomalia. Reimplementamos o GraphIDS do zero em PyTorch e o avaliamos nos quatro datasets do artigo (NF-UNSW-NB15 e NF-CSE-CIC-IDS2018, v2 e v3) e no NF-ToN-IoT-v3. Com três seeds, 7 das 8 métricas ficam a até dois desvios-padrão do artigo (PR-AUC de 0,9983 no NF-UNSW-NB15-v3). No NF-ToN-IoT-v3, o desempenho cai para PR-AUC de 0,6872 e F1 macro de 0,6408, porque varreduras e boa parte dos ataques web e de negação de serviço são reconstruídos tão bem quanto o tráfego normal.**

***Index Terms*—detecção de intrusão, redes neurais de grafos, aprendizado auto-supervisionado, autoencoder mascarado, NetFlow**

I. Introdução

Investigamos a detecção de intrusões a partir de registros de fluxo (NetFlow): dado o tráfego entre hosts, decidir quais fluxos pertencem a um ataque, sem depender de rótulos de ataque. O problema é relevante porque ataques novos surgem continuamente e rotular tráfego é caro; NIDS supervisionados reconhecem bem o que viram no treino, mas generalizam mal para ataques desconhecidos [1], [2].

As soluções existentes têm limitações claras. Detectores de anomalia clássicos e autoencoders sobre fluxos isolados [3]–[5] ignoram a estrutura de comunicação entre hosts, que caracteriza varreduras e ataques distribuídos. GNNs como o E-GraphSAGE [6] exploram essa estrutura, mas são supervisionadas. O Anomal-E [7] dispensa rótulos com um pré-treino contrastivo, porém separa representação e detecção e depende de amostras negativas artificiais. Por isso o artigo de referência [8] propõe o GraphIDS, que treina de ponta a ponta um GNN e um autoencoder Transformer para reconstruir representações de tráfego benigno, de modo que os embeddings são otimizados para a própria detecção.

Este trabalho reproduz o GraphIDS e testa sua generalização num ambiente novo. As contribuições são:

- uma reimplementação do zero, comentada e testada, em PyTorch puro, sem DGL nem PyTorch Geometric [9];
- a reprodução nos quatro datasets do artigo com três seeds, com 7 das 8 métricas a até dois desvios-padrão dos valores publicados;
- a primeira avaliação do GraphIDS no NF-ToN-IoT-v3, com análise por categoria de ataque que expõe limitações do método.

II. Trabalhos relacionados

Modelagem mascarada. BERT [10], MAE [11] e GraphMAE [12] aprendem representações reconstruindo partes mascaradas da entrada; como anomalias são mal reconstruídas, a ideia foi adaptada à detecção de anomalias, por exemplo em logs [13]. Nenhum deles trata dados em que a informação está nas arestas.

GNNs supervisionadas. O GraphSAGE [14] é indutivo e o E-GraphSAGE [6] acrescenta features de aresta, classificando fluxos NetFlow com alto desempenho. Friji *et al.* [15] modelam fluxos como nós e Zhou *et al.* [16] detectam botnets só pela topologia. Todos exigem tráfego rotulado.

Métodos auto-supervisionados. O Anomal-E [7] pré-treina um E-GraphSAGE com Deep Graph Infomax [17] e aplica detectores clássicos aos embeddings, em duas etapas independentes. O Kitsune [5] e o SAFE [18] usam autoencoders sem contexto topológico, e o SAFE perde muito desempenho nos datasets maiores (Tabela II).

III. Modelo de ameaça

O defensor observa a rede num ponto de agregação que exporta NetFlow para um coletor, sobre o qual roda o GraphIDS (Figura 1). O atacante é externo ou controla um host interno comprometido. O tráfego é um grafo dirigido $G = (V, E)$: cada nó é um IP e cada aresta (u, v) é um fluxo com features $\mathbf{x}_{uv} \in \mathbb{R}^F$. Com os graus $d^-(v) = |\{(u, v) \in E\}|$ e $d^+(u) = |\{(u, v) \in E\}|$, as famílias de ataque dos datasets deixam assinaturas distintas:

- *Varredura* (Reconnaissance, scanning): $d^+(a) \gg \mathbb{E}[d^+]$ para o atacante a, com fluxos curtos para muitos destinos ou portas.
- *DoS/DDoS*: origens S enviam fluxos a uma vítima v em alta taxa, com $|S| = 1$ (DoS) ou $|S| \gg 1$ (DDoS), elevando $d^-(v)$.
- *Força bruta* (SSH, FTP, password): muitos fluxos quase idênticos, $\mathbf{x}_{av} \approx \mathbf{x}'_{av}$, no mesmo par e porta.

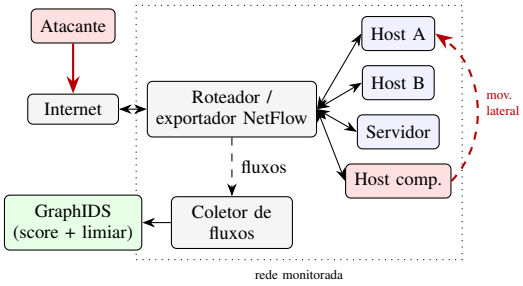


Figura 1. Modelo de ameaça: ataques externos ou a partir de um host comprometido geram fluxos que passam pelo ponto de observação e são analisados pelo GraphIDS.

- *Aplicação* (injection, XSS, Exploits): conexão igual à de um cliente legítimo; a diferença só aparece indiretamente nas features.
- *Comprometimento* (Infiltration, Backdoor, Bot, ransomware, MITM): um host interno ganha arestas novas (comando e controle, varredura interna); quando imita o tráfego normal, a assinatura é fraca.

Premissas. (P1) o atacante alcança a rede ou controla um host interno; (P2) o tráfego de ataque passa pelo ponto de observação; (P3) os IPs identificam os hosts de forma estável (sem NAT ou *spoofing* sistemático); (P4) o treino contém só tráfego benigno, sem envenenamento; (P5) o atacante não adapta o tráfego para evadir o modelo; (P6) há uma pequena validação rotulada para escolher o checkpoint e o limiar.

IV. Sistema proposto pelo artigo de referência

O GraphIDS [8] tem quatro componentes (Figura 2).

Construção do grafo. *Entrada:* tabela de fluxos; *saída:* um grafo dirigido por partição (treino, validação, teste), com IPs como nós e as demais colunas NetFlow como features de aresta.

E-GraphSAGE. *Entrada:* um lote de arestas e sua vizinhança amostrada; *saída:* um embedding por fluxo. Com um salto e agregação por média [6]:

$$\mathbf{a}_w = \frac{1}{|\mathcal{N}_K(w)|} \sum_{(z,w)\in\mathcal{N}_K(w)} \mathbf{x}_{zw}, \tag{1}$$

$$\mathbf{h}_w = \mathrm{ReLU}\left(\mathbf{W}_n \mathbf{a}_w + \mathbf{b}_n\right), \tag{2}$$

$$\mathbf{h}_{uv} = \mathrm{Dropout}\left(\mathbf{W}_e \left[\mathbf{h}_u \,\|\, \mathbf{h}_v\right] + \mathbf{b}_e\right), \tag{3}$$

em que $\mathcal{N}_K(w)$ são até $K = 32.768$ arestas que chegam em w. O modelo é indutivo (gera embeddings para hosts não vistos) e o salto único basta para capturar o contexto local; o padrão global fica com o Transformer.

Autoencoder Transformer com máscara. *Entrada:* embeddings embaralhados em janelas de até 512 fluxos (64 janelas por lote); *saída:* reconstruções $\hat{\mathbf{h}}_i$. Os embeddings são projetados para d_{model}, passam por encoder e decoder (o decoder usa a mesma sequência como consulta e faz atenção cruzada sobre o encoder) e voltam ao espaço do GNN. Não há codificação posicional, pois o modelo aprende coocorrência, não sequência. No treino, uma máscara simétrica

Algoritmo 1: Treino do GraphIDS

Input: grafos de treino (benigno) e de validação
Output: parâmetros θ e limiar τ^*

```
for época = 1, ..., 100 do
    foreach lote de arestas do treino do
        h ← E-GraphSAGE com vizinhança amostrada;
        agrupar em janelas de 512 e sortear a máscara M;
        ĥ ← Transformer(h; M);
        atualizar GNN e Transformer com ∇L_MSE (AdamW);
    salvar θ se a PR-AUC de validação melhorou;
    if 20 épocas sem melhora then parar;
τ* ← arg max_τ F1_macro na validação;
```

$M_{ij} = M_{ji} \sim \mathrm{Bernoulli}(\rho)$, com $\rho = 0{,}15$, desliga ligações de atenção:

$$\mathrm{Att} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{A}\right)\mathbf{V}, \quad A_{ij} = \begin{cases} -\infty, & M_{ij} = 1 \\ 0, & M_{ij} = 0 \end{cases} \tag{4}$$

Nenhum token é removido: a máscara é um *dropout* estruturado que obriga a reconstruir a partir de contexto parcial, e é desligada na inferência.

Score, perda e limiar. O score é o erro quadrático de reconstrução, e a perda é sua média sobre as posições válidas:

$$s_i = \left\|\mathbf{h}_i - \hat{\mathbf{h}}_i\right\|_2^2, \qquad \mathcal{L}_{MSE} = \frac{1}{n_{valid}} \sum_{i=1}^{n_{valid}} s_i. \tag{5}$$

O gradiente atravessa o Transformer e o GNN (Algoritmo 1), alinhando representação e detecção; a regularização forte no GNN (weight decay de 0,6) evita o colapso para embeddings triviais. O limiar τ^* é o que maximiza o F1 macro entre 500 valores igualmente espaçados nos scores de validação, e é aplicado ao teste sem consultá-lo.

V. Solução proposta pela equipe

A proposta de melhoria é opcional e não foi realizada: o esforço foi dedicado à reimplementação, à reprodução e à avaliação no novo dataset. A Seção VIII indica as modificações que os resultados sugerem.

VI. Metodologia

A. Dados do artigo de referência

O artigo usa as versões NetFlow do UNSW-NB15 [19] e do CSE-CIC-IDS2018 [20], publicadas pela Universidade de Queensland [21]. Elas foram escolhidas por virem de ambientes distintos e terem um conjunto padronizado de features com os IPs, o que permite montar os grafos [22]. A v2 tem 43 features e a v3 acrescenta 10 temporais [23]. O UNSW-NB15 representa uma rede pequena (44 hosts) com nove categorias de ataque; o CSE-CIC-IDS2018, uma infraestrutura grande na AWS, com 14 categorias (força bruta, DoS/DDoS, Bot, Infiltration e ataques web). Nossas estatísticas (Tabela I) batem com a Tabela 1 do artigo.

Formação dos conjuntos. Seguimos o artigo: valores inválidos viram zero; IPs e, nos v3, timestamps saem das features;

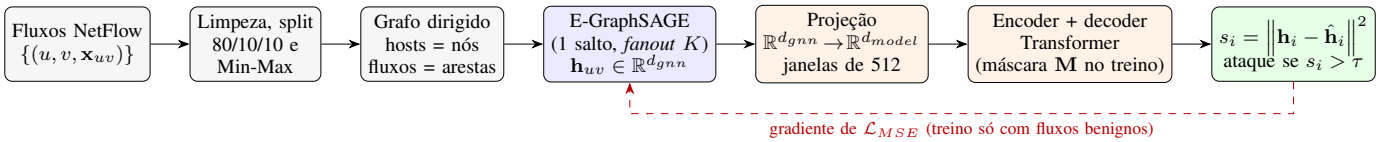


Figura 2. Arquitetura do GraphIDS. O gradiente do erro de reconstrução atualiza juntos o E-GraphSAGE e o Transformer.

Tabela I
DATASETS COMPLETOS E FRAÇÃO ESTRATIFICADA USADA NOS EXPERIMENTOS.

Dataset	Fluxos	Hosts	Ataques	Cat.	Amostra
UNSW-NB15-v2	2.390.275	44	3,98%	9	100%
UNSW-NB15-v3	2.365.424	44	5,40%	9	100%
CIC-IDS2018-v2	18.893.708	255.042	11,95%	14	20%
CIC-IDS2018-v3	20.115.529	205.801	12,93%	14	20%
ToN-IoT-v3	27.520.260	19.555	38,98%	9	20%

no CSE-CIC-IDS2018 usa-se uma amostra de 20% estratificada por categoria (como no artigo); o split é estratificado em 80/10/10; os ataques do treino são descartados; e o Min-Max é ajustado no treino benigno. As Tabelas III e IV trazem as contagens por categoria (seed 1). Nos UNSW completos, os ataques descartados do treino somam 76.042 (v2) e 102.154 (v3) fluxos.

B. Outro conjunto de dados: NF-ToN-IoT-v3

Escolhemos o NF-ToN-IoT-v3, versão NetFlow do TON_IoT [24], por três motivos: tem as mesmas 53 features v3, então o pipeline roda sem adaptação e a comparação isola o efeito do ambiente; representa um ambiente IoT/IIoT, diferente das redes corporativas do artigo; e traz ataques pouco cobertos pelo artigo (ransomware, injection, XSS, MITM, password), com 38,98% de ataques. Essa prevalência é a PR-AUC de um classificador aleatório, contra 0,04 a 0,13 nos datasets do artigo. Pela escala (27,5 milhões de fluxos), usamos a mesma amostra de 20% e o mesmo protocolo (Tabela V).

C. Métricas e experimentos

Usamos as métricas do artigo: PR-AUC (*average precision*, $\mathrm{AP} = \sum_k (R_k - R_{k-1}) P_k$) e F1 macro no limiar τ^*, além da taxa de detecção por categoria, $\mathrm{DR}_c = \mathrm{TP}_c / N_c$, e da taxa de falso positivo, $\mathrm{FPR} = \mathrm{FP}/N_{benigno}$ [25]. Cada dataset foi rodado com as seeds 1, 2 e 3 e os hiperparâmetros da Tabela 14 do artigo; o NF-ToN-IoT-v3 herdou a configuração do NF-UNSW-NB15-v3, sem ajuste. O critério de reprodução é ficar a até dois desvios-padrão do valor publicado. Os baselines vêm da Tabela 3 do artigo, sem reexecução. Os experimentos rodaram em GPUs NVIDIA A100 do cluster SLURM do CIn-UFPE, com PyTorch 2.14.1. Em vez de usar DGL, nossa amostragem de vizinhança pré-calcula a média da Eq. (1).

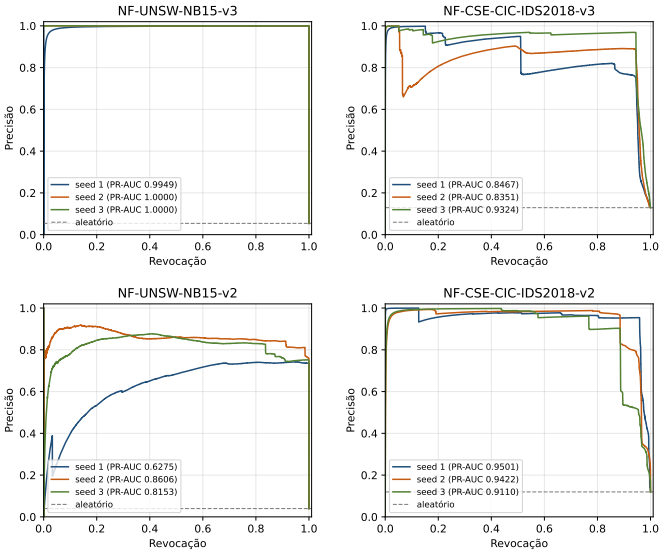


Figura 3. Curvas precisão-revocação no teste, uma por seed.

VII. RESULTADOS E DISCUSSÕES

A. Solução do artigo nos dados do artigo

A Tabela II compara nossa reprodução com o artigo e seus baselines. O critério é atendido em 7 das 8 métricas. A exceção é a PR-AUC do NF-UNSW-NB15-v3 (0,9983 contra 0,9998): apenas 0,0015 abaixo, mas fora dos dois desvios porque o desvio publicado é de 0,0007. No NF-CSE-CIC-IDS2018-v2 superamos o artigo (0,9344 contra 0,9201). As maiores diferenças vêm da variância entre seeds: no NF-UNSW-NB15-v2, uma seed teve PR-AUC de 0,6275 e as outras, 0,8606 e 0,8153.

A reprodução confirma a conclusão do artigo: nos v3, o GraphIDS supera com folga os baselines externos (no CSE-CIC-IDS2018-v3, 0,8714 de PR-AUC contra 0,6294 do SAFE e 0,2555 do Anomal-E), o que mostra o ganho de otimizar os embeddings para a detecção. Nos v2, fica próximo do Anomal-E.

Por categoria (Figura 3 e Tabela VIII). No NF-UNSW-NB15-v3 todas as categorias são detectadas em 100%, com FPR de 0,003%. No v2, todas passam de 99%, mas a FPR de 1,4% gera cerca de 3.200 falsos positivos para 9.506 ataques no teste e derruba a precisão, o que explica a PR-AUC menor, como o artigo também observa. No CSE-CIC-IDS2018, a perda se concentra na Infiltration (7% e 27% de detecção, contra 18% e 25% no artigo), cujo tráfego imita o normal, e no Bot, detectado por completo em algumas seeds e quase

Tabela II
DESEMPENHO NO TESTE (MÉDIA ± DESVIO ENTRE SEEDS). AS LINHAS ACIMA DA NOSSA SÃO DA TABELA 3 DE GUERRA *et al.* [8].

	Datasets v3		Datasets v2	
Modelo / métrica	UNSW-NB15	CIC-IDS2018	UNSW-NB15	CIC-IDS2018
SAFE – PR-AUC	$0,8946 \pm 0,0279$	$0,6294 \pm 0,0923$	$0,2044 \pm 0,0267$	$0,5222 \pm 0,1896$
SAFE – F1 macro	$0,9236 \pm 0,0086$	$0,4662 \pm 0,0016$	$0,5815 \pm 0,0164$	$0,4684 \pm 0,0001$
Anomal-E – PR-AUC	$0,9032 \pm 0,0041$	$0,2555 \pm 0,0383$	$0,7489 \pm 0,0074$	$0,9287 \pm 0,0265$
Anomal-E – F1 macro	$0,9459 \pm 0,0009$	$0,6709 \pm 0,0394$	$0,9156 \pm 0,0217$	$0,9410 \pm 0,0161$
SimpleAE – PR-AUC	$0,9996 \pm 0,0007$	$0,8458 \pm 0,0498$	$0,7864 \pm 0,0608$	$0,9310 \pm 0,0126$
SimpleAE – F1 macro	$0,9838 \pm 0,0321$	$0,9223 \pm 0,0201$	$0,8680 \pm 0,1579$	$0,9450 \pm 0,0092$
GraphIDS (artigo) – PR-AUC	$0,9998 \pm 0,0007$	$0,8819 \pm 0,0347$	$0,8116 \pm 0,0367$	$0,9201 \pm 0,0238$
GraphIDS (artigo) – F1 macro	$0,9961 \pm 0,0084$	$0,9447 \pm 0,0213$	$0,9264 \pm 0,0217$	$0,9431 \pm 0,0131$
GraphIDS (nosso) – PR-AUC	$0,9983 \pm 0,0029$	$0,8714 \pm 0,0531$	$0,7678 \pm 0,1236$	$0,9344 \pm 0,0207$
GraphIDS (nosso) – F1 macro	$0,9999 \pm 0,0001$	$0,9252 \pm 0,0435$	$0,9232 \pm 0,0038$	$0,9567 \pm 0,0179$

Tabela III
FLUXOS POR CATEGORIA E PARTIÇÃO NOS DATASETS NF-UNSW-NB15 (SEED 1). O TREINO SÓ TEM BENIGNOS.

	UNSW-NB15-v2			UNSW-NB15-v3		
Categoria	Treino	Val.	Teste	Treino	Val.	Teste
Benign	1.836.178	229.522	229.522	1.790.185	223.773	223.773
Analysis	0	230	230	0	123	122
Backdoor	0	217	217	0	466	466
DoS	0	579	580	0	598	598
Exploits	0	3.155	3.155	0	4.275	4.275
Fuzzers	0	2.231	2.231	0	3.381	3.382
Generic	0	1.656	1.656	0	1.965	1.965
Reconnaissance	0	1.278	1.278	0	1.707	1.708
Shellcode	0	142	143	0	238	238
Worms	0	17	16	0	16	16
Ataques (total)	0	9.505	9.506	0	12.769	12.770
Total	1.836.178	239.027	239.028	1.790.185	236.542	236.543
Hosts com arestas	43	40	41	44	43	42

Tabela IV
FLUXOS POR CATEGORIA E PARTIÇÃO NOS DATASETS NF-CSE-CIC-IDS2018, APÓS A AMOSTRA DE 20% (SEED 1).

	CIC-IDS2018-v2			CIC-IDS2018-v3		
Categoria	Treino	Val.	Teste	Treino	Val.	Teste
Benign	2.661.690	332.711	332.712	2.802.339	350.293	350.293
Bot	0	2.862	2.862	0	4.154	4.154
Brute Force -Web	0	43	43	0	33	32
Brute Force -XSS	0	19	18	0	9	10
DDOS attack-HOIC	0	21.617	21.618	0	20.646	20.646
DDOS attack-LOIC-UDP	0	42	42	0	69	69
DDoS attacks-LOIC-HTTP	0	6.146	6.146	0	5.772	5.772
DoS attacks-GoldenEye	0	554	555	0	1.226	1.226
DoS attacks-Hulk	0	8.653	8.653	0	2.001	2.002
DoS attacks-SlowHTTPTest	0	283	282	0	2.111	2.111
DoS attacks-Slowloris	0	190	190	0	721	721
FTP-BruteForce	0	519	518	0	7.735	7.734
Infilteration	0	2.327	2.328	0	3.763	3.763
SQL Injection	0	9	8	0	9	8
SSH-Bruteforce	0	1.899	1.900	0	3.769	3.770
Ataques (total)	0	45.163	45.163	0	52.018	52.018
Total	2.661.690	377.874	377.875	2.802.339	402.311	402.311
Hosts com arestas	101.953	29.939	29.893	88.817	27.550	27.448

nada em outras, sinal de que o limiar fica muito perto dos seus scores.

Custo. A Tabela VI mostra que nosso modelo tem cerca de um décimo dos parâmetros reportados e treina em minutos em

Tabela V
FLUXOS POR CATEGORIA E PARTIÇÃO NO NF-TON-IOT-V3, APÓS A AMOSTRA DE 20% (SEED 1).

Categoria	Treino	Validação	Teste	Total
Benign	2.686.754	335.844	335.845	3.358.443
Backdoor	0	4.068	4.067	8.135
ddos	0	82.825	82.825	165.650
dos	0	4.069	4.069	8.138
injection	0	7.635	7.636	15.271
mitm	0	121	120	241
password	0	31.895	31.896	63.791
ransomware	0	80	79	159
scanning	0	27.179	27.180	54.359
xss	0	56.689	56.689	113.378
Ataques (total)	0	214.561	214.561	429.122
Total	2.686.754	550.405	550.406	3.787.565
Hosts com arestas	6.355	1.540	1.529	7.406

Tabela VI
CUSTO COMPUTACIONAL (MÉDIA $\pm$ DESVIO), COMPARADO COM A TABELA 2 DO ARTIGO. A MEMÓRIA DE GPU FICOU ENTRE 0,9 E 1,3 GB.

	Parâmetros (M)		Treino (min)	
Dataset	nosso	artigo	nosso	artigo
UNSW-NB15-v3	0,100	0,874	$8,7 \pm 3,5$	$37,8 \pm 19,2$
CIC-IDS2018-v3	0,059	0,572	$8,7 \pm 4,1$	$87,6 \pm 35,4$
UNSW-NB15-v2	0,093	0,867	$2,5 \pm 0,9$	$23,4 \pm 16,2$
CIC-IDS2018-v2	0,058	0,570	$7,8 \pm 6,5$	$60,0 \pm 33,0$
ToN-IoT-v3	0,100	–	$5,4 \pm 2,7$	–

Tabela VII
TAXA DE DETECÇÃO (%) POR CATEGORIA NO NF-TON-IOT-V3.

Categoria	Fluxos	Taxa (%)
Benign (FPR)	335.845	$9,9 \pm 7,3$
Backdoor	4.067	$100,0 \pm 0,0$
ddos	82.825	$49,8 \pm 36,0$
dos	4.069	$36,4 \pm 25,7$
injection	7.636	$67,7 \pm 17,3$
mitm	120	$34,4 \pm 36,5$
password	31.896	$43,4 \pm 29,6$
ransomware	79	$100,0 \pm 0,0$
scanning	27.180	$4,2 \pm 5,0$
xss	56.689	$40,5 \pm 20,5$

vez de horas, com memória de GPU semelhante e inferência de 0,24 a 0,82 μs por fluxo (2,27 a 5,09 μs no artigo). Não identificamos a causa da diferença de parâmetros, mas ela não afetou o desempenho.

B. Solução do artigo no NF-ToN-IoT-v3

O GraphIDS obtém PR-AUC de $0{,}6872 \pm 0{,}0192$ e F1 macro de $0{,}6408 \pm 0{,}1488$. A PR-AUC é 1,8 vez a de um classificador aleatório (0,3898, a prevalência no teste), contra 6,7 a 19,3 vezes nos datasets do artigo. Não há resultados publicados neste dataset para comparação. Backdoor e ransomware são sempre detectados, mas scanning (4%) e MITM, DoS, XSS, password e DDoS (34% a 50%) ficam bem abaixo, com FPR de 9,9% (Tabela VII).

Tabela VIII
FPR E CATEGORIAS COM DETECÇÃO MÉDIA ABAIXO DE 99% NO TESTE (MÉDIA $\pm$ DESVIO ENTRE SEEDS).

Dataset	Categoria	Fluxos	Taxa (%)
UNSW-NB15-v3	Benign (FPR)	223.773	$0,0 \pm 0,0$
	demais categorias		$\geq 99{,}0$
CIC-IDS2018-v3	Benign (FPR)	350.293	$3,0 \pm 2,0$
	Bot	4.154	$66,7 \pm 47,1$
	Infilteration	3.763	$26,7 \pm 4,2$
UNSW-NB15-v2	Benign (FPR)	229.522	$1,4 \pm 0,0$
	demais categorias		$\geq 99{,}0$
CIC-IDS2018-v2	Benign (FPR)	332.712	$0,8 \pm 0,4$
	Bot	2.862	$33,3 \pm 47,1$
	Infilteration	2.328	$7,2 \pm 9,4$

Os resultados apontam três causas. (1) *Desalinhamento entre reconstrução e detecção:* a perda cai durante todo o treino, mas a PR-AUC de validação atinge o máximo entre as épocas 1 e 6 e depois cai (Figura 4b); o modelo passa a reconstruir bem também os ataques, e o scanning chega a ter score menor que o benigno (Figura 4a). (2) *Ataques e benignos compartilham hosts:* há só 6.355 hosts para 2,7 milhões de fluxos de treino e, pelas Eqs. (1)–(3), o embedding de um fluxo depende das médias de vizinhança dos dois hosts, em que a feature do próprio fluxo entra diluída; o t-SNE (Figura 4c) mostra categorias bem agrupadas nas features brutas e muito mais misturadas nos embeddings. (3) *Sem ajuste de hiperparâmetros*, ao contrário do artigo, que otimiza cada dataset. A alta prevalência de ataques também enfraquece a premissa de que anomalias são raras.

C. Proposta da equipe

Não há resultados de uma versão modificada, pois a melhoria não foi realizada.

VIII. CONCLUSÃO E TRABALHOS FUTUROS

Reimplementamos o GraphIDS e reproduzimos seus resultados nos quatro datasets do artigo, com 7 das 8 métricas a até dois desvios-padrão e com um modelo menor e mais rápido. No NF-ToN-IoT-v3, o desempenho cai bastante (PR-AUC de 0,6872).

As principais limitações do método são: o limiar e o *early stopping* dependem de validação rotulada; a reconstrução não garante separação quando os ataques são frequentes ou regulares; o embedding do fluxo depende só da vizinhança dos hosts, o que prejudica ataques que compartilham hosts com o tráfego normal (Infiltration, ToN-IoT); e não há avaliação contra mudanças de distribuição, evasão ou envenenamento (P4 e P5). Do nosso lado, usamos três seeds, não reexecutamos os baselines e não ajustamos o NF-ToN-IoT-v3.

Como trabalhos futuros, propomos: ajustar os hiperparâmetros do NF-ToN-IoT-v3 pela PR-AUC de validação; concatenar $\mathbf{x}_{uv}$ a $[\mathbf{h}_u \| \mathbf{h}_v]$ na Eq. (3); usar limiar e critério de parada não supervisionados; e avaliar a robustez a ataques adaptativos.

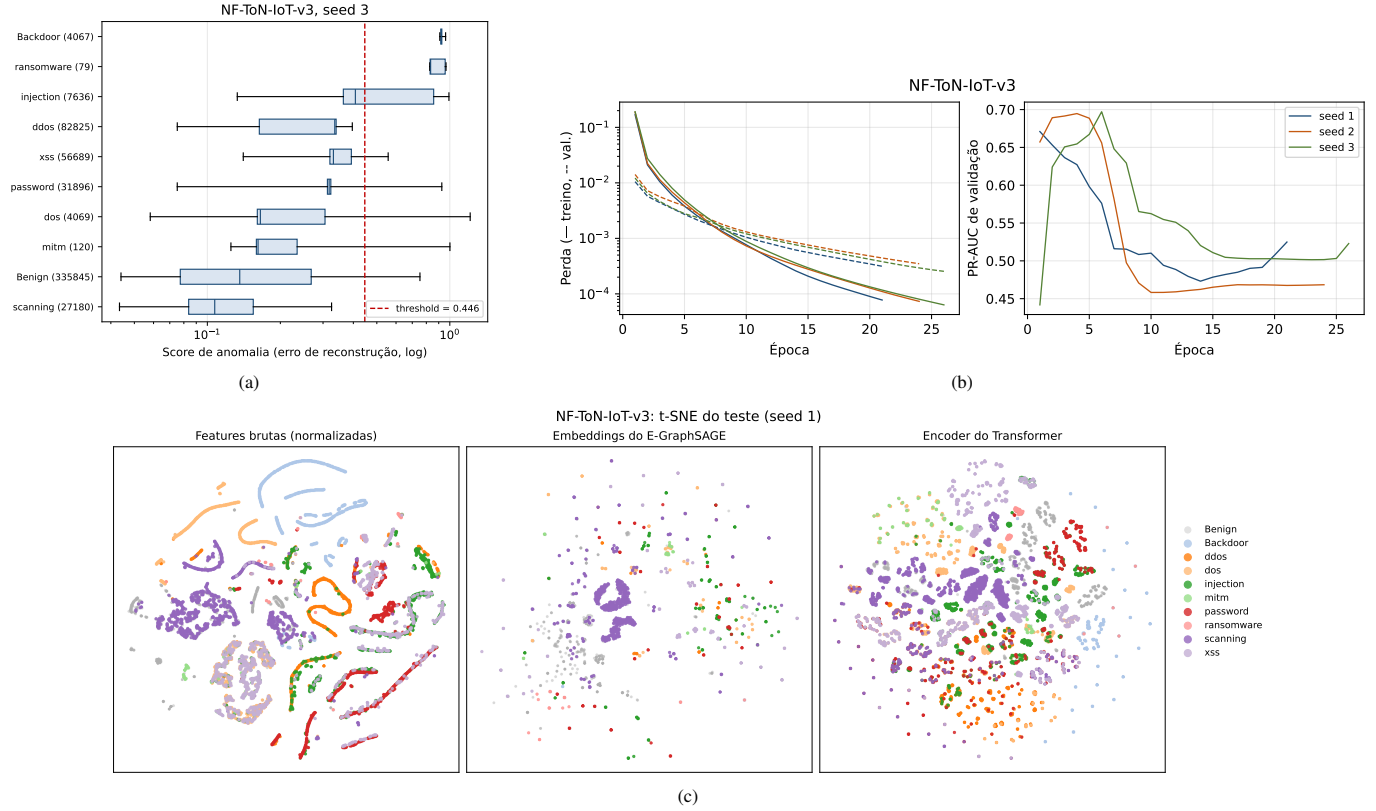


Figura 4. NF-ToN-IoT-v3: (a) score por categoria no teste, com o limiar (seed 3); (b) perdas e PR-AUC de validação por época; (c) t-SNE [26] das features, dos embeddings do E-GraphSAGE e da saída do encoder (seed 1).

Referências

[1] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.

[2] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset," *IEEE Access*, vol. 9, pp. 22 351–22 370, 2021.

[3] Z. He, X. Xu, and S. Deng, "Discovering cluster-based local outliers," *Pattern Recognition Letters*, vol. 24, no. 9, pp. 1641–1650, 2003.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[5] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[6] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022, pp. 1–9.

[7] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, vol. 258, p. 110030, 2022.

[8] L. Guerra, T. Chapuis, G. Duc, P. Mozharovskyi, and V.-T. Nguyen, "Self-supervised learning of graph representations for network intrusion detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, arXiv:2509.16625. Código: https://github.com/lorenzo9uerra/GraphIDS.

[9] Equipe 10 – CIN0114, "GraphIDS: reimplementação e avaliação," https://github.com/rxavier1904/projeto-GraphIDS-equipe-10, 2026.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.

[11] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 15 979–15 988.

[12] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang, "GraphMAE: Self-supervised masked graph autoencoders," in *Proc. 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 594–604.

[13] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.

[14] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[15] H. Friji, A. Olivereau, and M. Sarkiss, "Efficient network representation for GNN-based intrusion detection," in *Applied Cryptography and Network Security (ACNS)*. Springer Nature Switzerland, 2023, pp. 532–554.

[16] J. Zhou, Z. Xu, A. M. Rush, and M. Yu, "Automating botnet detection with graph neural networks," AutoML for Networking and Systems Workshop, MLSys, 2020, arXiv:2003.06344.

[17] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[18] E. Li, Z. Shang, O. Gungor, and T. Rosing, "SAFE: Self-supervised anomaly detection framework for intrusion detection," arXiv:2502.07119, 2025.

[19] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6.

[20] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating

a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[21] University of Queensland, "Machine learning-based NIDS datasets," https://staff.itee.uq.edu.au/marius/NIDS_datasets/, acesso em: nov. 2026.

[22] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

[23] M. Luay, S. Layeghy, S. Hosseininoorbin, M. Sarhan, N. Moustafa, and M. Portmann, "Temporal analysis of NetFlow datasets for network intrusion detection systems," arXiv:2503.04404, 2025.

[24] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021.

[25] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[26] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.